

Sistemas de tipos

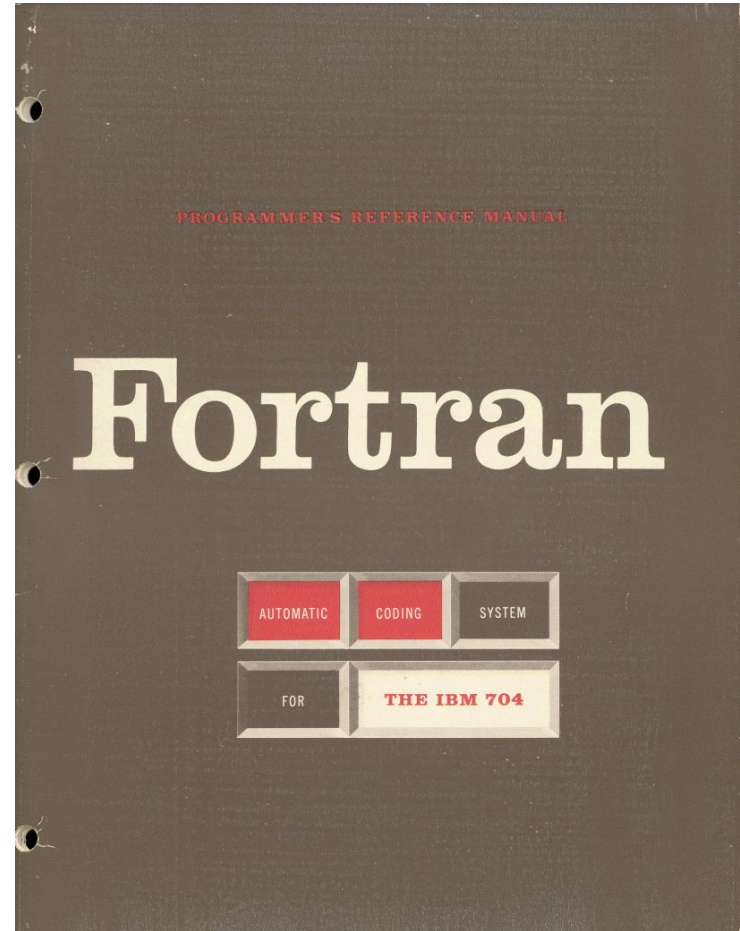
En donde hacemos un repaso de la noción de tipo, y de las distintas estrategias que podemos tener para definirlos.

¿Qué es un tipo?

Los orígenes

The image features a solid red background. A white diagonal line runs from the bottom-left corner towards the top-right corner, dividing the space into two triangular regions. The text "Los orígenes" is positioned in the upper-left triangular region, rendered in a white, classic serif typeface.

Fortran (1956)



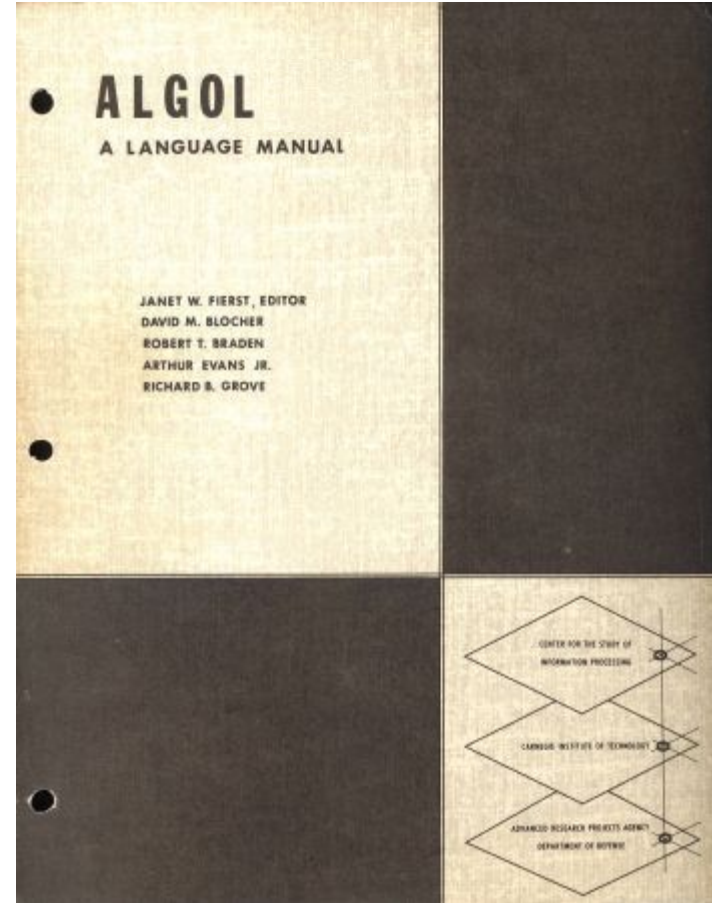
Formal Rules for Forming Expressions. By repeated use of the following rules, all permissible expressions may be derived.

1. Any fixed point (floating point) constant, variable, or subscripted variable is an expression of the same mode. Thus 3 and I are fixed point expressions, and ALPHA and A(I,J,K) are floating point expressions.
2. If SOMEF is some function of n variables, and if E, F, , H are a set of n expressions of the correct modes for SOMEF, then SOMEF (E, F, , H) is an expression of the same mode as SOMEF.
3. If E is an expression, and if its first character is not + or -, then +E and -E are expressions of the same mode as E. Thus -A is an expression, but +-A is not.
4. If E is an expression, then (E) is an expression of the same mode as E. Thus (A), ((A)), (((A))), etc. are expressions.
5. If E and F are expressions of the same mode, and if the first character of F is not + or -, then

$$E + F$$
$$E - F$$
$$E * F$$
$$E / F$$

are expressions of the same mode.

ALGOL 58 (1958)



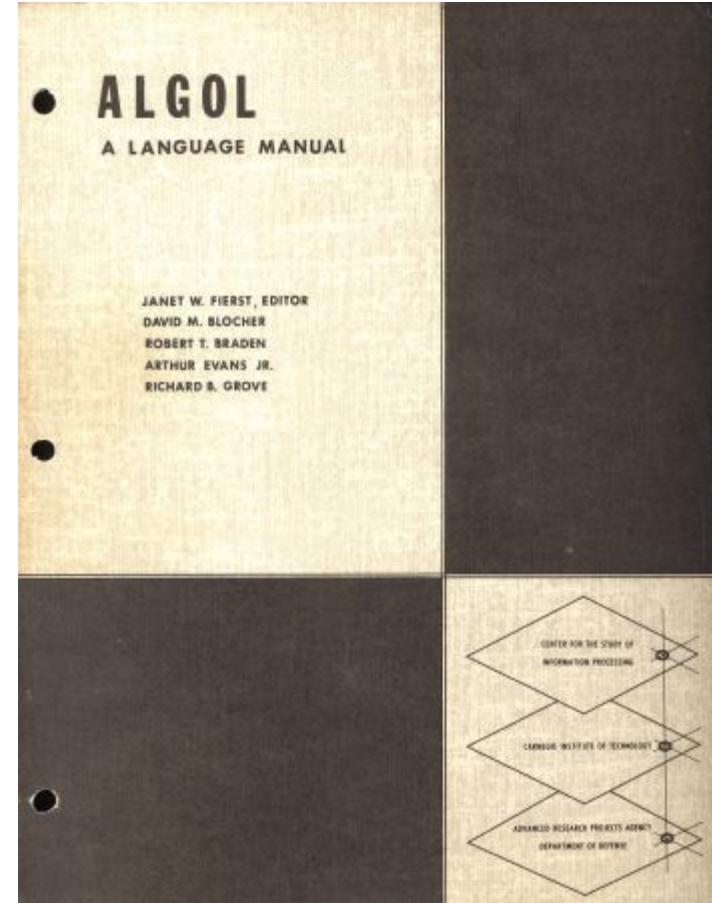
Variables (both simple and subscripted ones) designate arbitrary real numbers unless otherwise specified. However, certain declarations (cf. *type declarations*) may specify them to be of a special type, e.g., *integral*, or *Boolean*. Boolean (or logical) variables may assume only the two values *true* and *false*.

Type declarations

Type declarations serve to declare certain variables, or functions, to represent quantities of a given class, such as the class of integers or class of Boolean values.

Form: $\Delta \sim \textit{type} (I, I, \dots, I, I[, \dots, I[,], \dots, I[,], \dots)$ where *type* is a symbolic representative of some *type* declarator such as *integer* or *boolean*, and the *I* are identifiers. Throughout the program, the variables, or functions named by the identifiers *I*, are constrained to refer only to quantities of the type indicated by the declarator. On the other hand, all variables, or functions which are to represent other than arbitrary real numbers must be so declared.

ALGOL 60 (1960)



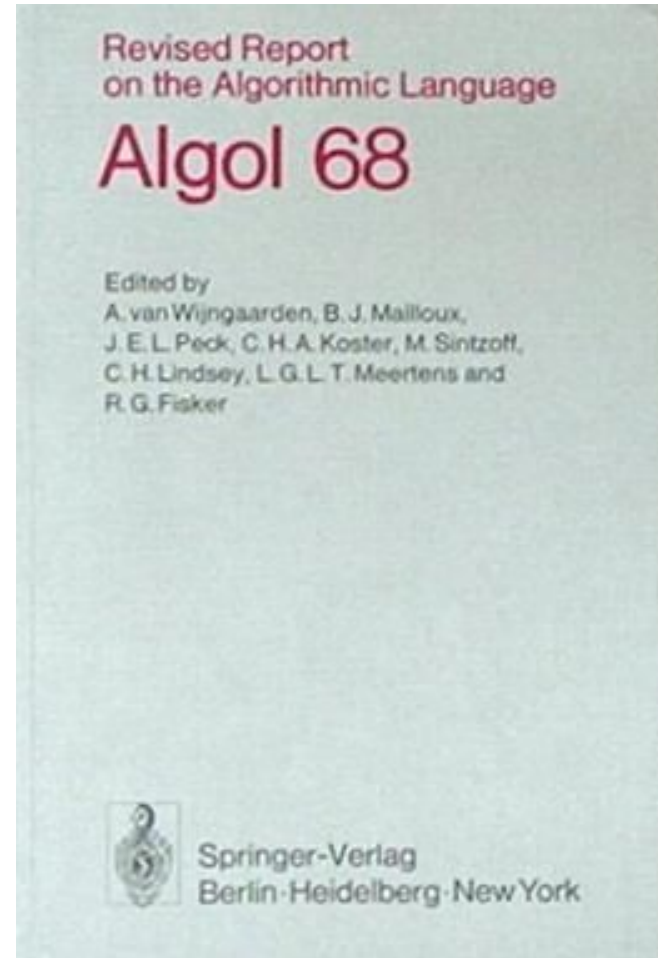
2.8. Values and types

A value is an ordered set of numbers (special case: a single number), an ordered set of logical values (special case: a single logical value), or a label.

Certain of the syntactic units are said to possess values. These values will in general change during the execution of the program. The values of expressions and their constituents are defined in section 3. The value of an array identifier is the ordered set of values of the corresponding array of subscripted variables (cf. section 3.1.4.1).

The various "types" (integer, real, Boolean) basically denote properties of values. The types associated with syntactic units refer to the values of these units.

ALGOL 68 (1968)



2.2.4. Modes and scopes

2.2.4.1. Modes

- a) Each instance {2.2.1} of a value is of one specific mode {1.1.6.c.vi} which is a terminal production of 'NONUNITED' {1.2.1.b}; furthermore, all instances of a given value other than nil {2.2.3.5.a} are of one same mode.
- b) The mode of a truth value (character, format) is 'boolean' ('character', 'format').
- c) The mode of an integer (a real number) of length number n is $(n - 1)$ times 'long' followed by 'integral' (by 'real').

Basic types

Data types in Algol 68 are called **MODE** s. The **MODE** s provided as standard are mostly unsurprising (see Table 1).

INT	Signed Integer. Single machine word.
REAL	Single-precision floating point number.
COMPL	Complex number.
BITS	Binary value, packed Booleans.
BYTES	Packed characters (provided for efficiency on architectures that are not byte-addressable, such as the ICL1900)
All of the above modes may be prefixed with LONG or LONG LONG to get double or higher precision, depending on the implementation.	
CHAR	Character. Implementation defined representation.
BOOL	Boolean value

Table 1

Additional **MODE** s are defined in terms of these simple types. A **MODE** declaration associates an identifier with a **MODE** as an alias for the full definition.



https://en.wikipedia.org/wiki/Hilbert%27s_problems
<https://es.wikipedia.org/wiki/Logicomix>

La teoría de conjuntos



Teoría de conjuntos

$S1 = \{1, 2\}$

$1 \in S1$ ✓

$2 \in S1$ ✓

$5 \in S1$ ✗

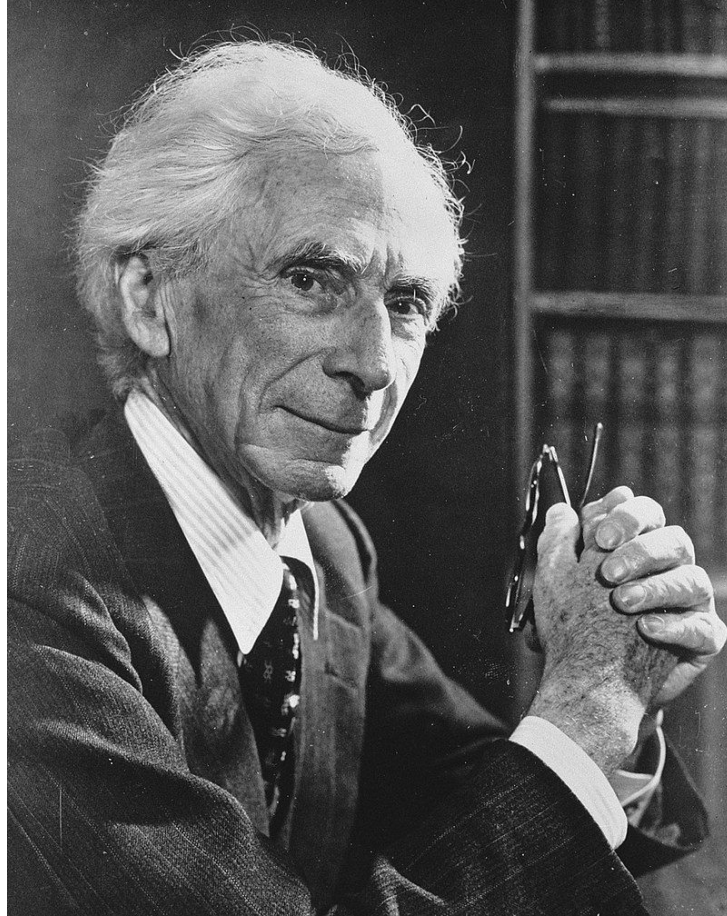
$S1 = \{ n \mid n = 1 \vee n = 2 \}$

$S2 = \{n \mid n.is_a?(Number) \}$

$1 \in S2$ ✓

$2 \in S2$ ✓

$5 \in S2$ ✓



Bertrand Russell -

https://commons.wikimedia.org/wiki/File:Bertrand_Russell_cropped.jpg

Teoría de conjuntos

Un conjunto puede contener a otros conjuntos...

$S = \{ c \mid c.is_a?(Set) \}$

$S1 \in S$ 

$S2 \in S$ 

E incluso, puede (o no) contenerse a sí mismo...

$S \in S$ 

Teoría de conjuntos

Consideremos el conjunto de todos los conjuntos que no se contienen a sí mismos:

$$W = \{ c \mid c.is_a?(Set) \wedge c \notin c \}$$

Entonces, por ejemplo:

$$S \in W \quad \text{X}$$

Ahora... W , se contiene a sí mismo?

$$W \in W \quad ?$$



<https://es.wikipedia.org/wiki/Logicomix>

WHITEHEAD
AND RUSSELL

PRINCIPIA
MATHEMATICA
VOLUME I

SECOND EDITION



CAMBRIDGE

WHITEHEAD
AND RUSSELL

PRINCIPIA
MATHEMATICA
VOLUME II

SECOND EDITION



CAMBRIDGE

WHITEHEAD
AND RUSSELL

PRINCIPIA
MATHEMATICA
VOLUME III

SECOND EDITION



CAMBRIDGE

PRINCIPIA
MATHEMATICA

By

ALFRED NORTH WHITEHEAD

AND

BERTRAND RUSSELL

VOLUME III

CAMBRIDGE UNIVERSITY PRESS

***54·43.** $\vdash :: \alpha, \beta \in 1 . \supset : \alpha \cap \beta = \Lambda . \equiv . \alpha \cup \beta \in 2$

Dem.

$\vdash . *54·26 . \supset \vdash :: \alpha = \iota'x . \beta = \iota'y . \supset : \alpha \cup \beta \in 2 . \equiv . x \neq y .$

$[*51·231] \qquad \qquad \qquad \equiv . \iota'x \cap \iota'y = \Lambda .$

$[*13·12] \qquad \qquad \qquad \equiv . \alpha \cap \beta = \Lambda \qquad (1)$

$\vdash . (1) . *11·11·35 . \supset$

$\vdash :: (\exists x, y) . \alpha = \iota'x . \beta = \iota'y . \supset : \alpha \cup \beta \in 2 . \equiv . \alpha \cap \beta = \Lambda \qquad (2)$

$\vdash . (2) . *11·54 . *52·1 . \supset \vdash . \text{Prop}$

From this proposition it will follow, when arithmetical addition has been defined, that $1 + 1 = 2$.

CHAPTER II

THE THEORY OF LOGICAL TYPES

THE theory of logical types, to be explained in the present Chapter, recommended itself to us in the first instance by its ability to solve certain contradictions, of which the one best known to mathematicians is Burali-Forti's concerning the greatest ordinal. But the theory in question is not wholly dependent upon this indirect recommendation: it has also a certain consonance with common sense which makes it inherently credible. In what follows, we shall therefore first set forth the theory on its own account, and then apply it to the solution of the contradictions.

V. *The Hierarchy of Functions and Propositions.*

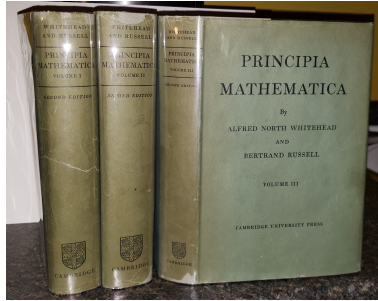
We are thus led to the conclusion, both from the vicious-circle principle and from direct inspection, that the functions to which a given object a can be an argument are incapable of being arguments to each other, and that they have no term in common with the functions to which they can be arguments. We are thus led to construct a hierarchy. Beginning with a and the other terms which can be arguments to the same functions to which a can be argument, we come next to functions to which a is a possible argument, and then to functions to which such functions are possible arguments, and so on.

Continúa...

Los teoremas de incompletitud



Kurt Gödel

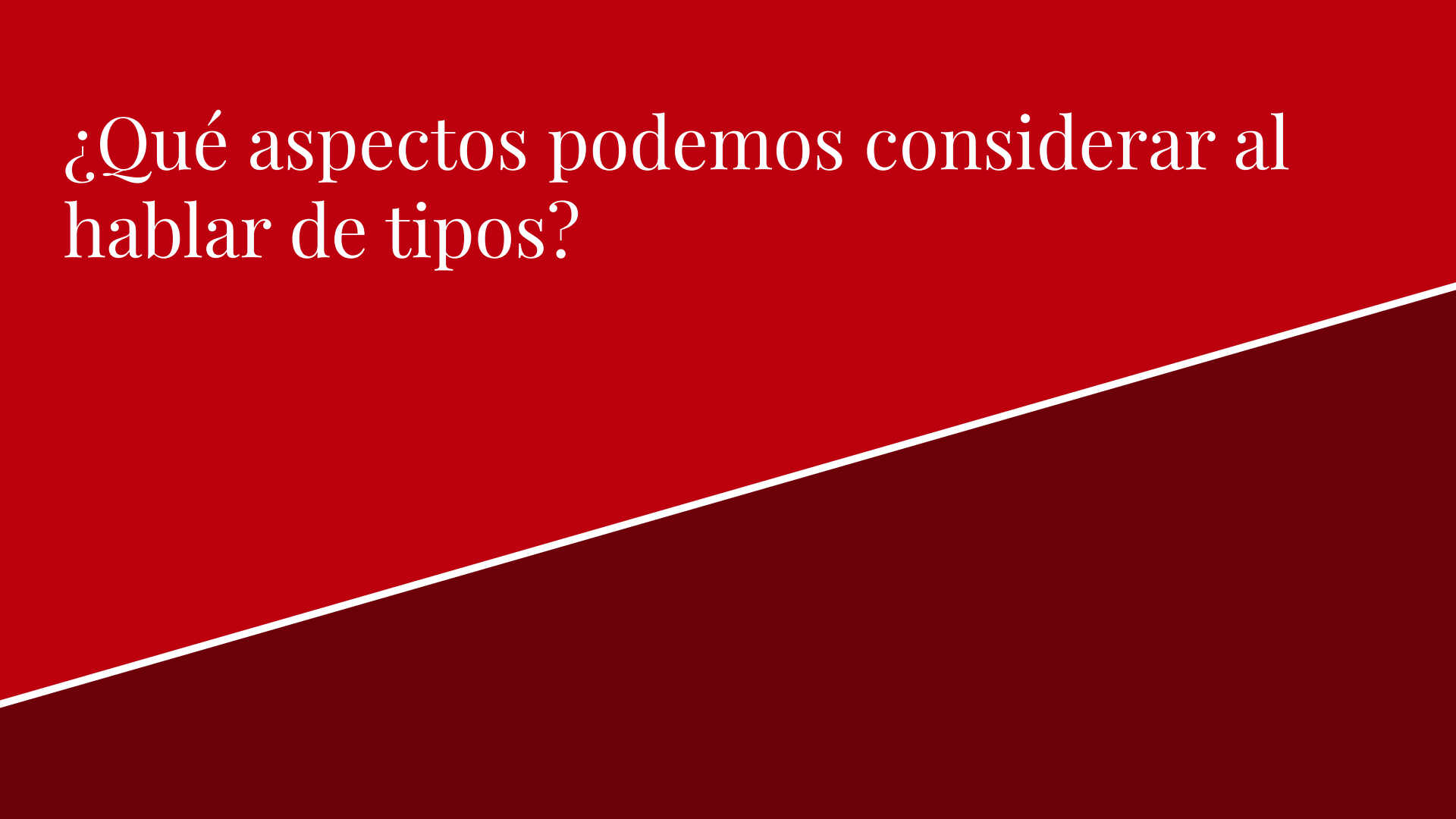


Axiomas
Reglas

$1 + 1 = 2$
 $1 + 2 > 3$
...

Números
Operaciones

¿Qué aspectos podemos considerar al hablar de tipos?

The image features a solid red background. A white diagonal line runs from the bottom-left corner towards the top-right corner, dividing the image into two triangular sections. The text is positioned in the upper-left triangular section.

¿Qué aspectos podemos considerar al hablar de tipos?

Aspecto operacional / conductual

¿Qué operaciones se pueden realizar sobre un valor? Por ejemplo, qué mensajes entiende un objeto.

Aspecto representacional

¿Cómo se representa un valor? (por ejemplo, ¿cómo se guarda un valor en la memoria?)

Aspecto constructivo / composicional

¿Cómo se compone un valor? ¿Cómo se puede construir/deconstruir? ¿Qué variantes posibles hay?

Categorización de los sistemas de tipos

Para ordenar mejor los diferentes esquemas de tipado que se utilizan en lenguajes orientados a objetos, vamos a establecer las siguientes categorías:

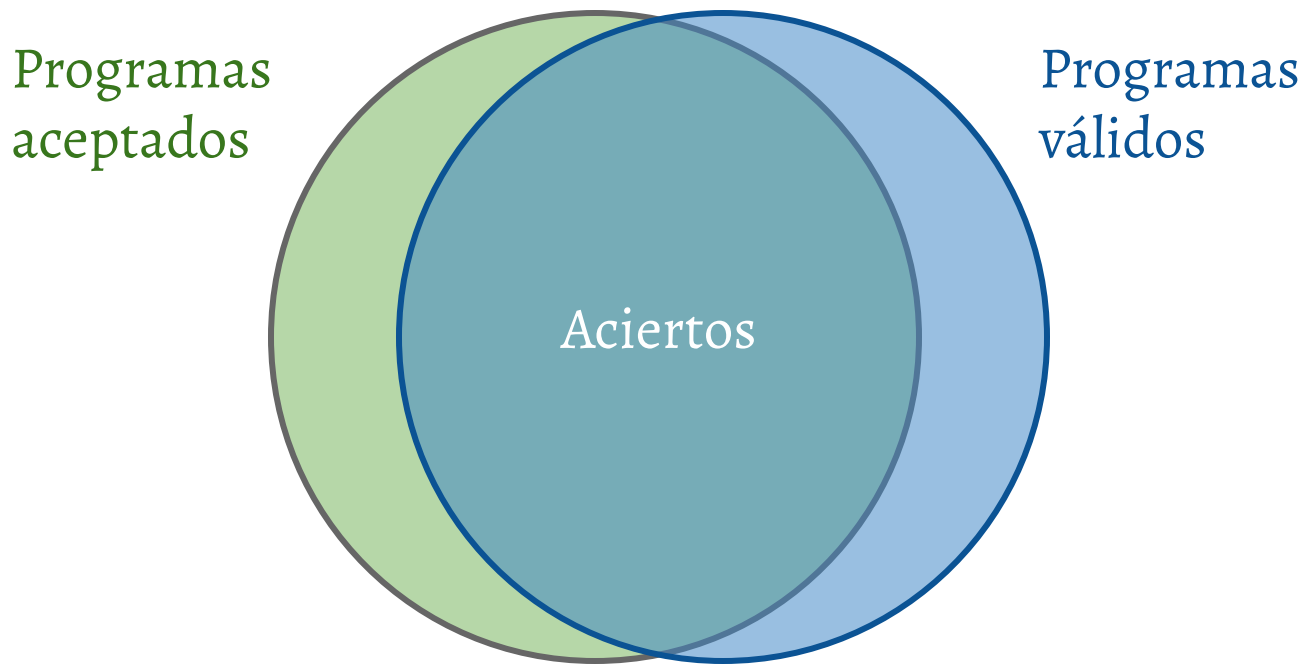
- Según el momento en el que se realiza el chequeo de tipos:
 - Estático (antes de la ejecución, por ejemplo en tiempo de compilación)
 - Dinámico (en tiempo de ejecución)
 - Ojo: estas categorías no son binarias (cf. *gradual typing*, *optional typing*, etc.).
- Según la forma en que se determina el tipo de una expresión o de una variable:
 - Explícito (estoy obligado a incluir anotaciones de tipos)
 - Implícito (no requieren que proveamos información de tipos)
 - Esto tampoco es binario, y además hay distintas variantes de inferencia (ej. inferencia global vs. local).

Categorización de los sistemas de tipos

- Según la forma en que se establece la relación entre un objeto y su tipo, así como las relaciones (de equivalencia y subtipado) entre tipos:
 - Nominal (requiere una declaración explícita)
 - Estructural (se basa en la estructura de los valores/tipos). ~“*duck typing*”
 - Muchos lenguajes mezclan ambos (ej. Scala, TypeScript, Go, ...).
- Según qué consecuencias tiene no pasar la verificación de tipos.
 - Seguro (*safe*), en donde se impide la realización de operaciones inválidas (ej. levantando una excepción o evitando la ejecución).
 - Inseguro (*unsafe*), en donde esto puede derivar en *comportamiento indefinido*.
 - Otras categorizaciones que se mezclan: fuerte/débil, *sound/unsound*.
- Según cuánta información de tipos está disponible durante la ejecución (*type erasure* / tipos reificados).
- ¿Hay lenguajes *no tipados*?

¿Puede haber un sistema de tipos perfecto? ¿Si el programa tipa, significa que funciona? ¿Si el programa funciona, significa que tipa?

Ningún sistema de tipos es perfecto



¿Qué pasa cuando empezamos a programar con un lenguaje estáticamente tipado?

Supongamos que solamente podemos
usar tipos simples, y veamos qué pasa...



Problema: ¿Cómo implementamos algunas cosas de manera general, sin tener código repetido?

En Ruby:

```
def id(x) = x
```

En Scala, ¿cómo lo tiparíamos?:

```
def id(x: Int) : Int = x  
def id(x: String) : String = x  
def id(x: Char) : Char = x  
  
def id(x: ???) : ??? = x
```

Problema: ¿Cómo implementamos algunas cosas de manera general, sin tener código repetido?

En Ruby:

```
def tamaño(xs) = xs.size
```

Pero en Scala, ¿cómo lo tiparíamos?:

```
def tamaño(xs: ListaDeInt)      : Int = xs.size  
def tamaño(xs: ListaDeString)   : Int = xs.size  
def tamaño(xs: ListaDeChar)     : Int = xs.size  
  
def tamaño(xs: ???) : Int = xs.size
```

Polimorfismo



Solución 1: Subtipado

Si tenemos que:

```
ListaDeInt <: Lista      ListaDeString <: Lista      ListaDeChar <: Lista
```

Entonces podemos escribir:

```
def tamaño(xs: Lista) : Int = xs.size
```

¿Y en el caso de `id`?

En Ruby:

```
def id(x) = x
```

En Scala:

```
def id(x: Any) : Any = x
```



Funciones/métodos con parámetros de tipo



Solución 2: Parámetros de tipo (*p. paramétrico*)

En Ruby:

```
def id(x) = x
```

En Scala:

```
def id[A] (x: A) : A = x
```

¿Podemos hacer algo parecido en el caso de las listas?

Supongamos que:

```
ListaDeInt <: Lista      ListaDeString <: Lista      ListaDeChar <: Lista
```

Entonces...

```
def tamaño[A] (xs: A) : Int = xs.size
```



Parámetros de tipo *acotados* (según subtipado)

Supongamos que:

```
ListaDeInt <: Lista    ListaDeString <: Lista    ListaDeChar <: Lista
```

Entonces...

```
def tamaño[A <: Lista](xs: A) : Int = xs.size
```

 ¿Nos sirve de algo en este caso resolverlo así?

¿Y cómo sería acá?

En Ruby:

```
def primero(xs) = xs[0]
```

En Scala, ¿cómo lo tiparíamos?:

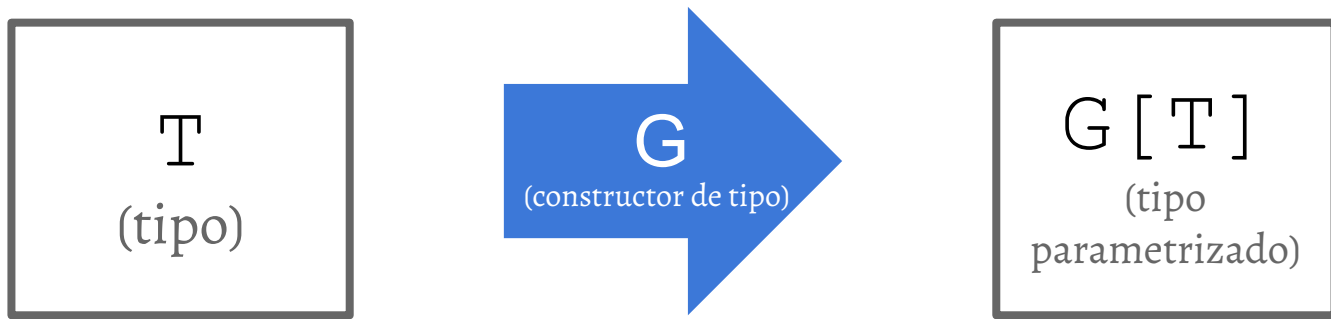
```
def primero(xs: ListaDeInt) : Int      = xs(0)
def primero(xs: ListaDeString) : String = xs(0)
def primero(xs: ListaDeChar) : Char    = xs(0)

def primero(xs: Lista) : ??? = xs(0)
```

Tipos parametrizados / clases genéricas

A decorative white diagonal line runs from the bottom-left corner towards the top-right corner, separating the red upper section from the dark red lower section.

¿Qué es un tipo parametrizado?



Usando clases genéricas / tipos parametrizados

Suponiendo que:

```
abstract class Lista[A] {  
    def apply(indice: Int): A  
    // ...  
}
```

Entonces podemos definir:

```
def primero[A](xs: Lista[A]) : A = xs(0)
```

Pero ahora ¿perdemos el subtipado?

Acaso sigue siendo cierto que existe un tipo `Lista[???]` tal que:

```
Lista[Int] <: Lista[???]
```

```
Lista[String] <: Lista[???]
```

```
Lista[Char] <: Lista[???]
```

No necesariamente...

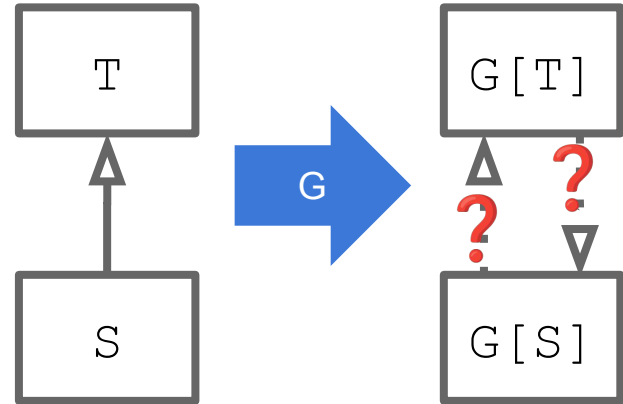
Varianza



Varianza

La varianza es la relación entre:

- el subtipado de los tipos parametrizados y
- el subtipado de sus argumentos de tipo.



Invarianza

Dado que:

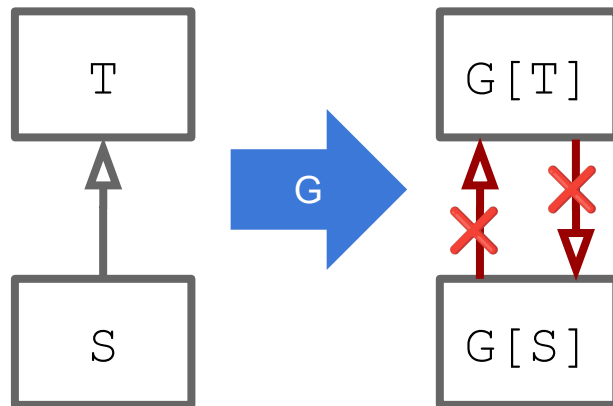
- ❑ $S <: T$ (S es un subtipo de T)
- ❑ G es un constructor de tipo

Entonces:

- ❑ $G[S] </: G[T]$
- ❑ $G[T] </: G[S]$

```
class T
class S extends T
class G[P]
```

```
val g1: G[T] = new G[S]() ❌
val g2: G[S] = new G[T]() ❌
```



Covarianza

Dado que:

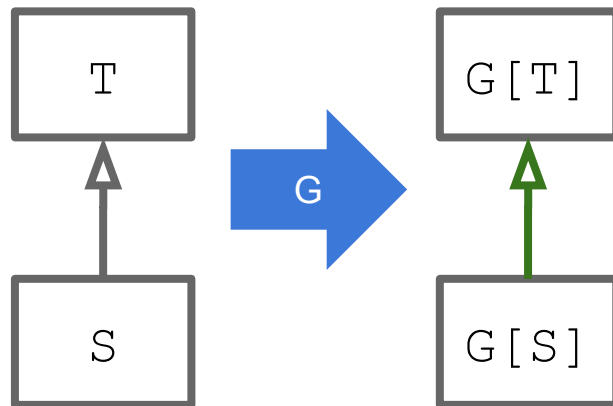
- ❑ $S <: T$ (S es un subtipo de T)
- ❑ G es un constructor de tipo

Entonces:

- ❑ $G[S] <: G[T]$

```
class T
class S extends T
class G[+P]
```

```
val g1: G[T] = new G[S]() ✓
```



Contravarianza

Dado que:

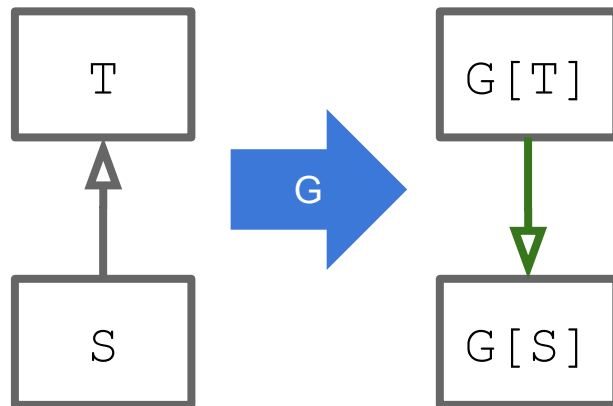
- ❑ $S <: T$ (S es un subtipo de T)
- ❑ G es un constructor de tipo

Entonces:

- ❑ $G[T] <: G[S]$

```
class T
class S extends T
class G[-P]
```

```
val g2: G[S] = new G[T]() ✓
```



Usando varianza

Suponiendo que:

```
abstract class Lista[+A] {  
    // ...  
}
```

Entonces por ejemplo:

```
Lista[Int] <: Lista[Any]
```

```
Lista[String] <: Lista[Any]
```

```
Lista[Char] <: Lista[Any]
```

Usando *wildcard arguments* (argumentos comodín)

Scala permite escribir “?” como parte del tipo, y esto significa “algún tipo, ni idea cuál”. Si hacemos eso, entonces (independientemente de la varianza) vale que:

```
Lista[Int] <: Lista[?]
```

```
Lista[String] <: Lista[?]
```

```
Lista[Char] <: Lista[?]
```

El tipo `Lista[?]` debe entenderse como equivalente a un *tipo existencial* ($\exists T. \text{Lista}[T]$).

Scala solía proveer sintaxis para definir tipos existenciales más complejos (ej. no anónimos, escribiendo `List[T] forSome { type T }`), pero fue eliminada en Scala 3.

Definiendo tamaño sin parámetros de tipo

Entonces, si lo siguiente sirve o no para todas las listas *depende de la varianza de Lista con respecto a su primer argumento de tipo*:

```
def tamaño[A <: Lista[Any]](xs: A) : Int = xs.size
```

Esto es equivalente, recibiendo un parámetro de tipo pero sin depender de la varianza:

```
def tamaño[A](xs: Lista[A]) : Int = xs.size
```

Y, en esta última opción, tamaño no recibe parámetros de tipo ni depende de la varianza:

```
def tamaño(xs: Lista[?]) : Int = xs.size
```

Y hay más...

¿Hasta qué punto se puede extender el sistema de tipos?



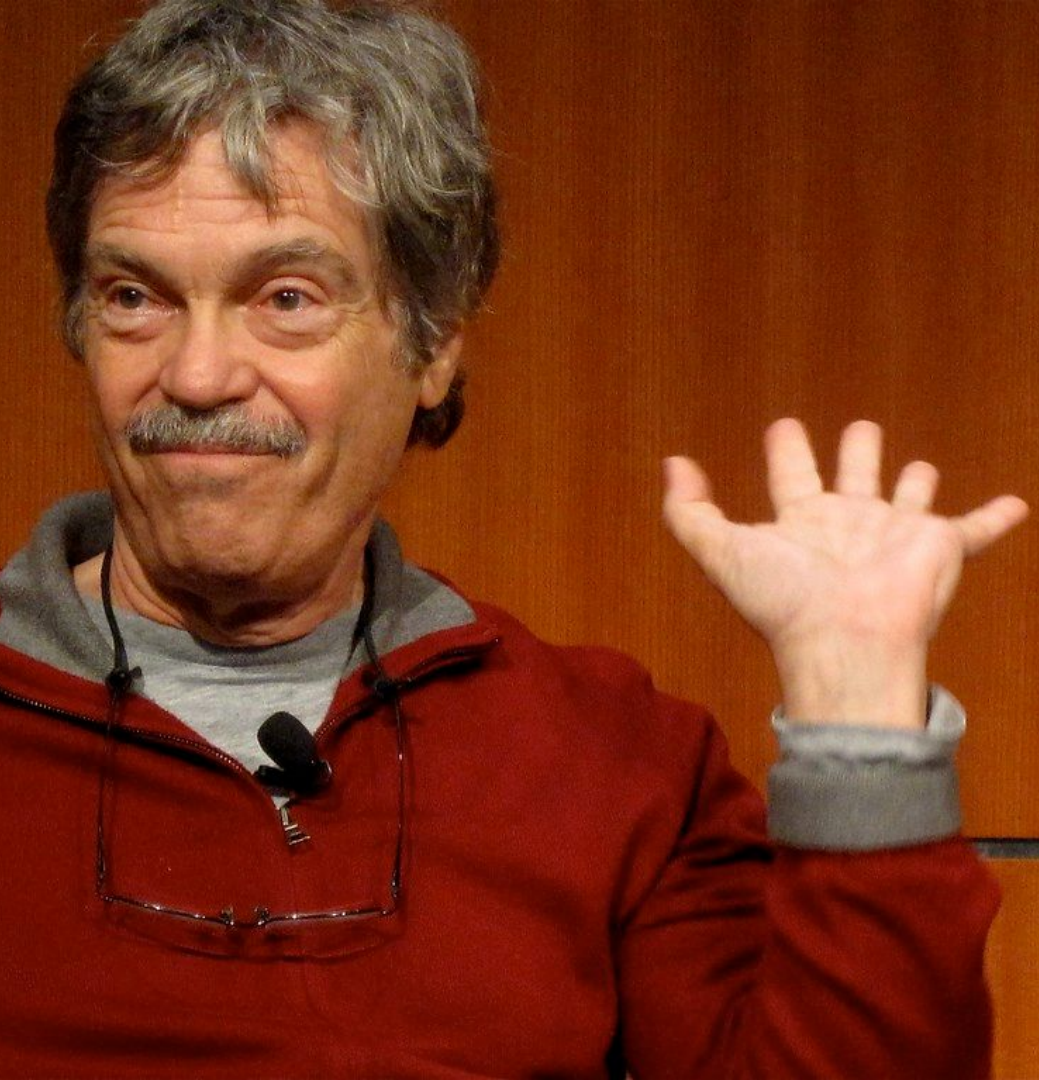
Continuará... (quizás)

Tener un lenguaje estáticamente tipado,
¿es una ventaja?



Desventajas de la verificación estática de tipos

- El lenguaje tiene más conceptos que entender para poder programar.
- Ciertas construcciones son muy difíciles de validar, por ejemplo las que tienen que ver con self-modification.
 - Si no tipar implica no compilar, no puedo programar desde el debugger!
- Es necesario proveer más información en el código. En consecuencia, muchos lenguajes estáticamente tipados son muy verbosos (aunque no todos).
- La validación puede llegar a rechazar programas válidos.
- La verificación de tipos puede llevar mucho tiempo de ejecución.



“I’m not against
types, but I don’t
know of any type
systems that aren’t a
complete pain,
so I still like
dynamic typing”
- Alan Kay

Ventajas de la verificación estática de tipos

- Verificación de propiedades de mi programa.
 - Esto se vuelve especialmente importante en el contexto de los LLMs (hay mucho para debatir).
- Refactorings seguros.
- Documentación.
- ...

¿Preguntas?